

## Full length article

## Time–torque coordinated optimization for trajectory planning of industrial robots

Zeyun Xiao, Danfeng Sun, Donglai Zhu, Yong Wang, Yi Yan, Huifeng Wu<sup>\*</sup>*School of Computer Science, Hangzhou Dianzi University, Hangzhou, 310018, Zhejiang, China**Zhejiang Key Laboratory of New Industrial Internet Control Technology, Hangzhou, 310018, Zhejiang, China*

## ARTICLE INFO

## Keywords:

Trajectory optimization  
Industrial robot  
Torque prediction  
Heuristic algorithm

## ABSTRACT

Trajectory optimization is vital for improving the operational efficiency and reliability of industrial robotic arms. However, increasing task execution speed frequently induces sharp fluctuations in joint torque, which elevates energy consumption, places excessive stress on actuators, and excites structural vibrations. To tackle these issues, this study introduces a unified time–torque optimization framework that combines predictive modeling with heuristic search. The framework adopts a neural network incorporating frequency-domain correlation and a delay-aware mechanism to model dynamic torque variations. Based on the predicted torque profiles, the robot's motion trajectory is parameterized by quintic polynomials, and a multi-objective loss function is constructed to jointly minimize execution time and torque variation rate. Particle Swarm Optimization (PSO) is employed to perform a global search for intermediate joint velocities and accelerations, improving convergence toward near-optimal solutions. Experiments on a six-axis industrial robotic platform demonstrate that the proposed method effectively reduces execution time and smooths torque transitions, confirming its practicality for industrial applications.

## 1. Introduction

In modern manufacturing systems, industrial robotic arms are widely used in tasks such as welding, assembly, and high-speed material handling, making them critical enablers of flexible and automated production [1,2]. Trajectory planning and optimization quality directly influences trajectory planning and optimization has a direct impact on task execution efficiency, mechanical wear, and overall system stability [3]. To boost productivity, modern industrial environments often impose strict constraints on trajectory execution time. However, inadequate acceleration profiles or suboptimal trajectory parameterization may cause considerable joint torque oscillations, inducing substantial load variations and diminished dynamic compliance. Such outcomes raise energy expenditure and mechanical shock, hasten equipment degradation, shorten service life, and can even compromise operational safety [4].

In recent years, both sampling-based and optimization-based methods have helped enhance trajectory efficiency and smoothness. Yet, dynamic torque fluctuations caused by inertia, gravity, and varying loads are still inadequately resolved, particularly in complex industrial scenarios involving frequent alterations in robot configuration and payload [5]. These nonlinear and uncertain factors are difficult to model analytically and often result in trajectory deviations and

reduced positioning accuracy [6]. To tackle these challenges, Ghediri et al. [7] proposed an adaptive Proportional–Integral–Derivative (PID) tuning strategy based on deep reinforcement learning, achieving robust tracking under disturbances. Kružíč et al. [8] utilized neural networks in conjunction with base-mounted force/torque sensors to estimate end-effector forces without load interference. Huo et al. [9] developed a dynamic interaction model featuring an observer-based adaptive controller for robust force regulation under uncertainty. Liu et al. [10] introduced a 9-degree-of-freedom (DOF) motion planning method integrating a refined improved dynamic programming (DP) algorithm with an SCA-GBFS hybrid for pose-aware trajectory optimization, validating its effectiveness through simulation and physical experiments on wheel hub surfaces. Lončarević et al. [11] presented a novel spatiotemporal approach for visual inspection trajectory generation from CAD models, merging iterative learning control and reinforcement learning to adjust robot speed along the path, which cut cycle time by 53% without sacrificing image clarity.

As modern industrial systems increasingly demand flexibility, precision, and efficiency, robotic trajectory optimization has grown into a multifaceted problem involving multiple objectives and constraints [12]. Common goals include minimizing execution time, reducing joint torque fluctuations [13], and curbing energy consumption [14,15], all

<sup>\*</sup> Corresponding author at: School of Computer Science, Hangzhou Dianzi University, Hangzhou, 310018, Zhejiang, China.

E-mail address: [whf@hdu.edu.cn](mailto:whf@hdu.edu.cn) (H. Wu).

while satisfying bounds on joint velocity, acceleration, and torque—resulting in a multi-objective nonlinear optimization problem. Recent research has explored a variety of strategies to address these challenges. For instance, Chen et al. [16] proposed a multi-objective model to improve motion smoothness and pose stiffness in robotic grinding. Meanwhile, Zhang et al. [17] developed a time-energy optimization method using an interior-point algorithm under pseudo-path constraints, lowering peak jerk, acceleration, and velocity by over 40%, 25%, and 12%. In addition, De et al. [13] employed Chebyshev polynomial representations for multi-DOF trajectory optimization with few design parameters, significantly reducing RMS joint torque. Furthermore, Fabris et al. [18] proposed an online optimization scheme for single-DOF systems that jointly minimizes time and energy consumption with high computational efficiency. Likewise, Lozer et al. [19] presented a multi-stage optimization framework for redundant manipulators that minimizes jerk by jointly optimizing time intervals and joint configurations, validated through extensive simulations and experiments on a 7-DOF robot. Additionally, Bendali et al. [20] introduced a smooth trajectory planning method based on multi-quadratic radial basis functions (MQ-RBF), which minimizes a multi-objective function of time and jerk while ensuring velocity, acceleration, and jerk continuity.

In summary, recent studies have made significant progress in enhancing trajectory smoothness, energy efficiency, and execution performance through multi-objective and energy-aware optimization methods. While these approaches have achieved remarkable improvements in geometric continuity and energy consumption, they primarily focus on position, velocity, or acceleration profiles, and rarely consider the temporal variation of joint torque that directly reflects dynamic stability during motion. Moreover, most existing torque estimation methods rely either on analytical dynamic models, which are difficult to generalize under nonlinear friction and varying payloads, or on strategies that depend on real-time sensor feedback, which limits their scalability and practical deployment in industrial environments. To address these limitations, this paper proposes a unified time–torque optimization framework for multi-degree-of-freedom industrial robots. By integrating a data-driven torque prediction network with trajectory optimization, the framework explicitly models the temporal evolution of joint torque, achieving coordinated improvements in execution efficiency, torque smoothness, and overall dynamic stability. The main contributions are summarized as follows:

- A dual-mode torque prediction model is proposed, accommodating both full (torque feedback included) and lite (motion data only) modes, and employing frequency-domain correlation and a delay-aware mechanism to capture dynamic torque variations.
- A joint time–torque objective is formulated, and PSO is applied to optimize intermediate joint velocities and accelerations under dynamic and kinematic constraints.
- Results from experiments on a six-axis industrial robotic platform demonstrate that the proposed method produces smoother torque profiles and achieves a notable reduction in execution time.

The paper is organized as follows. Section 2 provides a review of related work in industrial time-series modeling and trajectory optimization for robotic systems. Section 3 examines the dynamic stability problem and the limitations of classical dynamic modeling, providing the motivation for the proposed framework. Section 4 details the overall framework, including a frequency-domain torque prediction network and a joint time–torque optimization strategy. Section 5 presents experimental results based on real-world industrial tasks, which demonstrates the effectiveness and adaptability of the method. Finally, Section 6 summarizes the main contributions and outlines potential directions for future research.

## 2. Related work

This section reviews recent progress of industrial robotics in two key areas of industrial robotics: time-series modeling and multi-objective trajectory optimization. The first part traces the evolution from traditional statistical models to deep learning methods that incorporate frequency-domain representations, with applications in state prediction, anomaly detection, and adaptive control. The second part highlights trajectory planning strategies that aim to balance execution time, path smoothness, and system stability using diverse optimization techniques. Representative studies from both domains are summarized below.

### 2.1. Industrial time-series modeling

Time-series modeling is fundamental to intelligent manufacturing systems, which enables critical functions such as predictive control, anomaly detection, and data-driven decision-making. Common applications range from equipment health monitoring and energy optimization to production scheduling, and adaptive process control [21]. Traditional methods, such as AutoRegressive Integrated Moving Average (ARIMA) [22], Kalman filters [23], and Support Vector Regression (SVR) [24] have established the theoretical groundwork for industrial time-series modeling. Nevertheless, these methods often struggle with nonlinear dynamics, long-term dependencies, and periodic disturbances, limiting their effectiveness in complex industrial settings.

To better capture complex temporal dependencies, recurrent neural networks (RNNs) have been widely adopted, including variants such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU). For example, Wahid et al. [25] proposed a hybrid model that combines convolutional neural networks (CNN) and LSTM with skip connections, achieving robust fault detection in multivariate predictive maintenance. In a separate study, Li et al. [26] introduced a dynamic time-delay reconstruction (DTDR) mechanism that increased convergence speed and prediction accuracy in attention-based LSTM (ALSTM) networks.

In recent years, Transformer-based architectures have attracted considerable attention in industrial time-series forecasting because of their strong ability to capture long-range dependencies. The core component of these models is the self-attention mechanism [27], which enables global dependency modeling and addresses the vanishing gradient issue that often occurs in long sequences. This leads to greater stability and improved representational power in multi-step prediction tasks. For example, Autoformer [28] enhances temporal modeling by replacing standard attention with an auto-correlation mechanism, which explicitly captures periodic patterns, and further improves smoothness through a time-delay aggregation module. Flowformer [29] models temporal dependencies via continuous-time normalizing flows, offering an alternative view of temporal correlation while reducing memory overhead. Informer [30] proposes ProbSparse attention to sparsify the attention space, significantly accelerating inference on long sequences and making it suitable for industrial-scale applications. Reformer [31] reduces attention complexity through locality-sensitive hashing (LSH), thus achieving sub-quadratic memory and time requirements.

Transformer-based models have shown strong performance in various industrial applications, including predictive maintenance, energy consumption modeling, and production planning. For example, Wang et al. [32] proposed the ITSP model, which incorporates dynamic distribution information to improve adaptability to non-stationary time series by enabling dynamic pattern switching and applying residual connections. Jin et al. [33] developed the TST model, combining a time-series tokenizer with the Transformer architecture to increase compactness within feature classes and separability across different classes.

Despite recent advances, existing Transformer-based models face two key limitations in robotic applications. First, their focus on time-domain patterns limits their ability to capture multi-scale disturbances

in the frequency domain, reducing performance on non-periodic trajectories. Second, delay-aware mechanisms are often static and cannot adjust to dynamic delay variations. In contrast, our model incorporates frequency-domain correlation and a learnable delay aggregation module, enabling robust time–torque modeling under complex, non-repetitive conditions.

## 2.2. Trajectory optimization for robotic systems

Recent research aimed at improving robotic trajectory execution efficiency has primarily focused on the design and implementation of trajectory optimization algorithms. These methods are commonly grouped into two categories: sampling-based and optimization-based approaches [34]. Among the sampling-based methods, Rapidly-exploring Random Trees (RRT) [35] and Probabilistic Roadmaps (PRM) [36] are widely recognized for their strong global search capabilities and probabilistic completeness, making them well-suited for path planning in high-dimensional and complex environments. However, earlier versions of RRT did not sufficiently address concerns such as solution optimality and convergence speed [37], which has limited their effectiveness in certain industrial scenarios.

In contrast to sampling-based approaches, optimization-based trajectory planning explicitly incorporates dynamic parameters such as time, velocity, and acceleration. This approach allows for fine-grained control over trajectory smoothness and physical feasibility. By framing the task as a constrained nonlinear optimization problem — for example, subject to joint limits — these methods are well-adapted to for time-optimal [38] and energy-efficient [39] trajectory design. Commonly used optimization techniques include Sequential Quadratic Programming (SQP) [40], interior-point methods [41], and a variety of heuristic algorithms. Among heuristic optimization methods, Particle Swarm Optimization (PSO) [42] is widely used to tune velocity and acceleration profiles in polynomial or spline-based trajectories, which strikes an effective balance between computational efficiency and solution quality. Covariance Matrix Adaptation Evolution Strategy (CMA-ES) [43] and Differential Evolution (DE) [44] have also been applied to multi-objective trajectory planning tasks, successfully reconciling competing objectives such as execution time and energy consumption. Recent studies have confirmed the practical value of these heuristic approaches. For example, Ekrem et al. [45] applied PSO to generate time-optimal trajectories for six-degree-of-freedom robotic arms, while Yang et al. [46] combined PSO with the Dynamic Window Approach (DWA) to improve both path exploration and convergence in dynamically changing settings.

Although recent optimization-based approaches have enhanced trajectory design through heuristic and nonlinear methods, most remain limited in modeling the interaction between dynamic system responses and trajectory parameters. Specifically, torque fluctuations induced by variable loads are often frequently overlooked, leading to sub-optimal performance under real-world disturbances. Moreover, many methods treat optimization as a static process, lacking interaction with predictive feedback during execution. By contrast, our framework incorporates torque-aware modeling and closed-loop adjustment, thus enabling trajectories to adapt dynamically to changing physical conditions.

## 3. Problem statement

This section formulates the research problem that motivates the proposed time–torque coordinated optimization framework. Section 3.1 analyzes the dynamic stability issues commonly encountered by industrial robots and discusses their impact on motion precision and system performance. Section 3.2 then examines the limitations of classical analytical dynamic models based on the Euler–Lagrange formulation, revealing the challenges in accurately modeling friction, compliance, and load-dependent dynamics in real industrial environments. Together, these analyses highlight the need for a data-driven torque prediction model capable of providing reliable dynamic information for trajectory optimization.

### 3.1. Dynamic stability issues of the industrial robot

During robotic trajectory execution, even small fluctuations in motion can propagate as vibrations that deteriorate end-effector precision and overall system stability. Although a planned trajectory may appear smooth in position and velocity space, uncoordinated acceleration transitions or asynchronous multi-axis movements often induce dynamic oscillations within the mechanical structure. These oscillations accumulate over time, leading to positional deviations, angular misalignment, and control inaccuracies that compromise motion quality and repeatability.

Fig. 1 illustrates a representative robotic operation in which dynamic instability may cause small motion deviations during transportation and alignment. In this operation, the joint states recorded by the robot drive system are shown in Fig. 2, where joint positions and velocities remain comparatively smooth, whereas the corresponding acceleration and torque responses exhibit pronounced oscillations and sharp spikes. Such fluctuations arise from abrupt dynamic transitions or insufficient motion coordination among multiple joints, which may excite structural resonance, induce control lag, and cause transient instability. These torque-induced disturbances serve as a primary indicator of dynamic degradation, particularly in high-speed or precision robotic tasks, highlighting the sensitivity of motion stability to torque variation.

Excessive torque fluctuations are mainly attributed to inappropriate velocity and acceleration planning, which amplify the rate of torque variation across multiple joints. Such excessive torque changes not only generate impulsive forces that excite structural vibrations but also accelerate mechanical fatigue and wear. Consequently, enhancing dynamic stability requires trajectory profiles that are not only kinematically smooth but also torque-aware, minimizing the rate of torque change and ensuring consistent joint actuation throughout the motion.

### 3.2. Limitations of dynamic torque modeling

To achieve torque-aware trajectory optimization, it is essential to establish a dynamic relationship between joint motion states and the corresponding torque responses. A precise understanding of this relationship provides the basis for analyzing how velocity and acceleration planning affect torque variation and overall stability. Traditionally, joint torque estimation relies on analytical dynamic modeling, where the mechanical structure and motion parameters of the robot are used to derive torque expressions based on rigid-body dynamics.

As illustrated in Fig. 3, the overall process of mapping from the end-effector pose  $P$  to the joint angles  $\theta$ , and finally to the joint torque  $\tau$ , is presented. The transformation between the Cartesian (TCP) space and the joint space is generally formulated through forward and inverse kinematics, where the Jacobian matrix  $J(\theta)$  describes the relationship between joint velocities and end-effector motion. This process is typically represented using the Denavit-Hartenberg (D-H) convention, providing a unified framework for expressing the geometric and kinematic relationships among robot links. Based on these formulations, the torque of each joint can be analytically expressed as a function of position, velocity, and acceleration within the standard rigid-body dynamics framework:

$$\tau = M(\theta)\ddot{\theta} + C(\theta, \dot{\theta})\dot{\theta} + G(\theta) + F_f(\dot{\theta}) + K_d(\theta) + \tau_{\text{ext}} \quad (1)$$

Where:

- $M(\theta)$ : inertia matrix, representing torques required to accelerate joints;
- $C(\theta, \dot{\theta})$ : Coriolis and centrifugal terms, accounting for inter-joint dynamics;
- $G(\theta)$ : gravitational torque to maintain static equilibrium;
- $F_f(\dot{\theta})$ : friction torque, including static, kinetic, and viscous components;

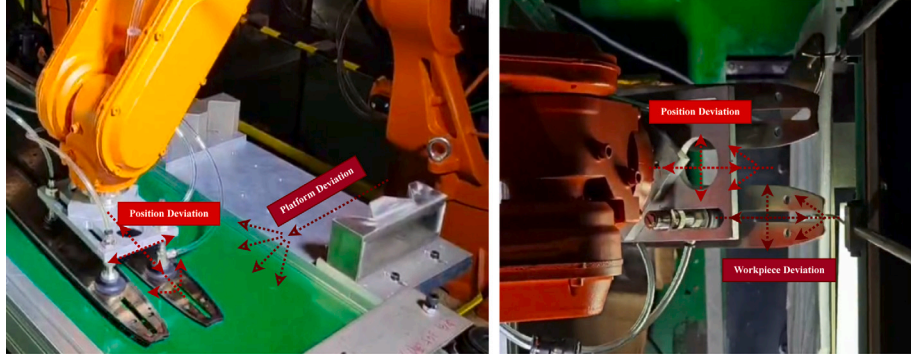


Fig. 1. Position and orientation deviations during real-world robotic transportation and alignment tasks.

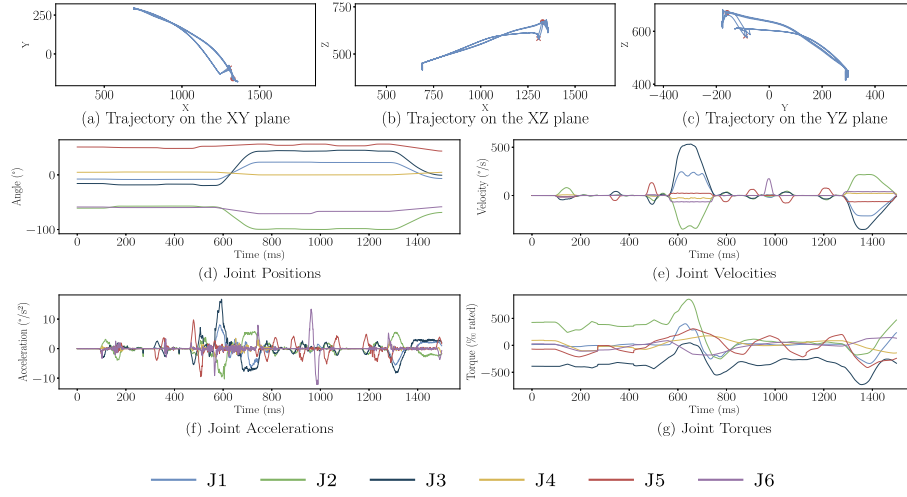


Fig. 2. Three-view trajectories of a typical industrial robotic handling task, and corresponding joint-level position, velocity, acceleration, and torque variation curves. Torque is expressed in per mille of the rated torque, measured directly from the motor output via EtherCAT at a 1 ms synchronization period.

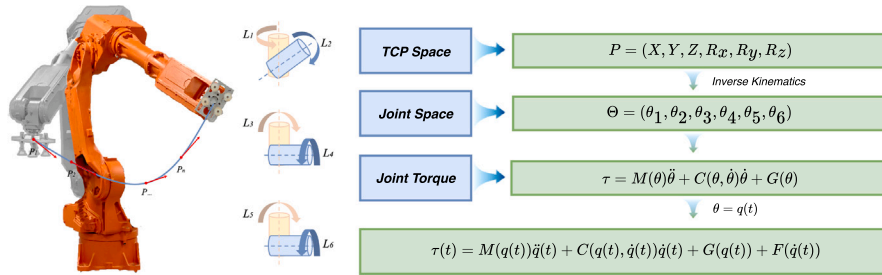


Fig. 3. Mapping relationships among the TCP pose, joint configuration, and joint torque in a classical rigid-body dynamic formulation.

- $K_q(\theta)$ : joint stiffness term, modeling elastic deformation and structural compliance;
- $\tau_{\text{ext}}$ : external disturbance torque, caused by payload variation or interaction forces.

Although the above formulation provides a comprehensive theoretical framework for describing joint dynamics, its practical implementation remains limited. Many parameters involved — such as friction coefficients, joint stiffness, and external disturbance torques — are difficult to identify accurately in real industrial environments. Moreover, these parameters may vary with robot type, structural materials, manufacturing tolerances, and service duration, further increasing modeling uncertainty. In addition, the dynamic coupling among multiple joints and the nonlinear characteristics of friction and compliance introduce additional complexity, especially under varying payloads and complex

trajectories. Such uncertainties can lead to noticeable deviations between analytically computed torques and measured responses, thereby reducing the reliability of purely model-based estimation in capturing transient and load-dependent torque variations.

Consequently, while the analytical dynamic model offers valuable insight into the mechanical structure and motion behavior of robotic systems, its reliance on precise parameter identification and idealized assumptions limits its robustness and adaptability in practice. This recognition motivates the development of data-driven approaches that can more effectively characterize complex torque behaviors under dynamic and uncertain operating conditions.

#### 4. Methodology

As shown in Fig. 4, the proposed trajectory optimization framework consists of two main modules: a delay-aware torque prediction model



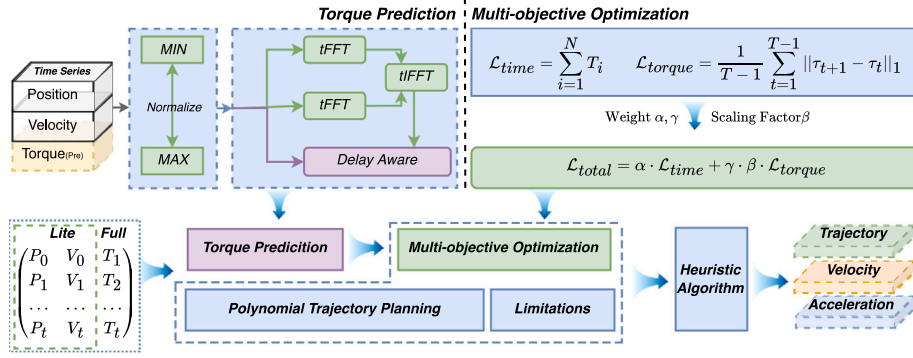


Fig. 4. Architecture of the trajectory planning model based on torque and time optimization.

and a heuristic multi-objective optimization module. The prediction model includes a delay-aware attention layer followed by a linear projection layer, enabling the capture of temporal variations in joint torque during robot motion. It operated in two configurations: a lite mode for evaluating the complete torque profile along the trajectory, and a full mode that integrates previous-step torque feedback to support online inference with enhanced robustness.

For trajectory generation, the robot's motion path is parameterized using quintic polynomial interpolation, which ensures smooth transitions in position, velocity, and acceleration. To achieve a balance between rapid task completion and smooth torque variation, the Particle Swarm Optimization (PSO) algorithm is applied to optimize velocity and acceleration values at intermediate waypoints. This scheme maintained execution efficiency while limiting torque variations across the motion.

#### 4.1. Dual prediction strategies: Full and lite settings

Two torque prediction strategies are introduced to meet varying demands for prediction accuracy and computational load: a high-accuracy full mode that utilizes torque feedback, and a lite mode alternative for trajectory planning without torque input. Both strategies use a sliding-window input structure but differ in input composition, inference behavior, and error sensitivity.

The full mode uses a high-frequency input window containing the most recent joint states and measured torque values. Given a window length  $T$ , the input tensor is defined as:

$$\mathbf{X}_{\text{full}} = [\mathbf{x}_{t-T+1}, \dots, \mathbf{x}_t] \in \mathbb{R}^{T \times d} \quad (2)$$

Each input vector is composed as:

$$\mathbf{x}_t = [\mathbf{q}_t, \dot{\mathbf{q}}_t, \boldsymbol{\tau}_t] \quad (3)$$

where  $\mathbf{q}_t$ ,  $\dot{\mathbf{q}}_t$ , and  $\boldsymbol{\tau}_t$  denote joint position, joint velocity, and measured joint torque at time  $t$ , respectively. Here,  $t$  refers to the discrete time index sampled at a fixed frequency.

Since the model relies on historical measured torque rather than recursive predictions, it avoids temporal error accumulation. This makes it suitable for feedback-sensitive control tasks. However, its dependency on real-time sensor input limits its applicability in offline optimization.

The lite mode also adopts a sliding-window structure but uses only simplified state information, such as joint positions and velocities. It follows the same temporal format as the full mode, but with a reduced input dimension  $d'$  due to the exclusion of torque history. Each input vector is defined as:

$$\mathbf{x}_t = [\mathbf{q}_t, \dot{\mathbf{q}}_t] \quad (4)$$

This simplified representation effectively reduces inference overhead, making it suitable for batch prediction in offline trajectory planning. Since each window is sampled independently from the trajectory, the model does not suffer from error propagation. However, the reduced feature set leads to slightly lower prediction accuracy compared to the full mode.

#### 4.2. Neural network-based torque prediction method

The proposed trajectory torque prediction model takes as input a sequence of recent joint states, including joint positions, velocities, and optionally torques, following the same sliding-window structure described in the previous section. The model outputs the predicted joint torques at the next time step. Specifically, the input sequence is denoted as  $\mathcal{X} \in \mathbb{R}^{B \times L \times C}$ , where  $B$  is the batch size,  $L$  is the sequence length, and  $C$  is the feature dimension, which is  $d$  in full mode and  $d'$  in lite mode.

The model first projects the input into two latent spaces using learnable weight matrices  $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{C \times D}$ , where  $D$  denotes the latent (embedding) feature dimension after the linear projection. This results in the query and key tensors:

$$\mathbf{Q} = \mathcal{X} \mathbf{W}_Q, \quad \mathbf{K} = \mathcal{X} \mathbf{W}_K, \quad \mathbf{Q}, \mathbf{K} \in \mathbb{R}^{B \times L \times D} \quad (5)$$

To capture global temporal dependencies in the frequency domain, a truncated Fast Fourier Transform (tFFT) is applied to both  $\mathbf{Q}$  and  $\mathbf{K}$ , retaining only the first  $k$  low-frequency components:

$$\hat{\mathbf{Q}} = \mathcal{F}_t(\mathbf{Q}), \quad \hat{\mathbf{K}} = \mathcal{F}_t(\mathbf{K}) \quad (6)$$

Next, element-wise complex multiplication is performed:

$$\hat{\mathbf{R}} = \hat{\mathbf{Q}} \odot \hat{\mathbf{K}} \quad (7)$$

The result is transformed back to the time domain via the inverse truncated FFT:

$$\mathbf{R} = \mathcal{F}_t^{-1}(\hat{\mathbf{R}}), \quad \mathbf{R} \in \mathbb{R}^{B \times L \times D} \quad (8)$$

To extract delay-dependent temporal features, the model computes an average correlation score across batches and feature dimensions:

$$\mathbf{r} = \frac{1}{B \cdot D} \sum_{b=1}^B \sum_{d=1}^D \mathbf{R}_{b, :, d}, \quad \mathbf{r} \in \mathbb{R}^L \quad (9)$$

The top- $K$  indices with the highest scores, denoted by  $\{\delta_1, \delta_2, \dots, \delta_K\}$ , are selected to represent dominant delay patterns. Attention weights are computed using the softmax function:

$$\alpha_k = \frac{\exp(r_{\delta_k})}{\sum_{j=1}^K \exp(r_{\delta_j})}, \quad k = 1, \dots, K \quad (10)$$

Each delay  $\delta_k$  is applied as a temporal shift on the value sequence  $\mathcal{V}$ , and a weighted sum produces the aggregated output:

$$\mathbf{H} = \sum_{k=1}^K \alpha_k \cdot \text{Roll}(\mathcal{V}, -\delta_k), \quad \mathbf{H} \in \mathbb{R}^{B \times L \times D} \quad (11)$$

here,  $\text{Roll}(\mathcal{V}, -\delta_k)$  denotes a cyclic temporal shift operator that shifts the sequence  $\mathcal{V}$  by  $\delta_k$  time steps along the temporal axis, enabling delay-aware feature aggregation. Finally, the aggregated features  $\mathbf{H}$  are concatenated with the original input  $\mathcal{X}$ , and passed through a linear projection to generate the predicted torque:

$$\boldsymbol{\tau} = \mathbf{W}_{\text{out}} [\mathbf{H} \parallel \mathcal{X}] + \mathbf{b}_{\text{out}}, \quad \boldsymbol{\tau} \in \mathbb{R}^{B \times L \times J} \quad (12)$$

$\mathbf{W}_{\text{out}}$  and  $\mathbf{b}_{\text{out}}$  denote the learnable weight matrix and bias term of the output projection layer, respectively, and  $J$  is the number of robot joints. This design allows the model to dynamically attend to delay patterns using a lightweight shift-based aggregation mechanism, without relying on deep encoder-decoder structures.

#### 4.3. Multi-objective trajectory optimization

After converting trajectory points from TCP space to joint space, a polynomial interpolation is applied to plan the motion of each joint independently, ensuring smooth transitions in position, velocity, and acceleration at both the start and the end of the motion. This satisfies the continuity and basic dynamic feasibility required for torque-aware trajectory optimization. Typical trajectory parameterizations in robotics include piecewise polynomials (e.g., cubic or quintic), time-scaling functions (e.g., trapezoidal/S-curve profiles), splines (e.g., B-spline/Bezier), and heuristic velocity profiles tailored to task constraints. These formulations provide well-defined expressions of joint position, velocity, and acceleration along the path.

Among these options, the piecewise fifth-degree (quintic) polynomial trajectory is adopted in this study. Quintic segments offer  $C^2$  continuity across waypoints and satisfy endpoint boundary conditions on position, velocity, and acceleration; this helps attenuate torque-rate fluctuations and improves stability during frequent start-stop operations. In addition, the analytic derivatives of quintic polynomials enable efficient evaluation within the optimization loop and are widely supported in industrial controllers, facilitating deployment. For each segment defined over  $[t_i, t_{i+1}]$ , the trajectory is expressed as:

$$\theta_i(t) = a_0 + a_1(t - t_i) + a_2(t - t_i)^2 + a_3(t - t_i)^3 + a_4(t - t_i)^4 + a_5(t - t_i)^5 \quad (13)$$

This polynomial includes six coefficients,  $a_0$  to  $a_5$ , which are uniquely determined by six boundary conditions on position, velocity, and acceleration at the start and end times. The corresponding boundary constraints can be expressed as:

- Position:

$$\theta_i^{(s)} = \theta_i(t_i), \quad \theta_i^{(e)} = \theta_i(t_{i+1}),$$

- Velocity:

$$v_i^{(s)} = \dot{\theta}_i(t_i), \quad v_i^{(e)} = \dot{\theta}_i(t_{i+1}),$$

- Acceleration:

$$a_i^{(s)} = \ddot{\theta}_i(t_i), \quad a_i^{(e)} = \ddot{\theta}_i(t_{i+1}).$$

The superscripts (s) and (e) denote the start and end conditions, respectively, for each boundary constraint. Based on quintic polynomial parameterization, we define a multi-objective loss function that jointly minimizes the total execution time and torque variation. The execution time is computed as the sum of durations across all trajectory:

$$\mathcal{L}_{\text{time}} = \sum_{i=1}^N T_i \quad (14)$$

To quantify torque stability, we define the torque rate as the mean absolute difference between consecutive predicted torque vectors:

$$\text{TorqueRate} = \frac{1}{T-1} \sum_{i=1}^{T-1} \|\tau_{i+1} - \tau_i\|_1$$

To improve numerical stability and mitigate the influence of large spikes, we apply a logarithmic transformation:

$$\mathcal{L}_{\text{torque}} = \log \left( 1 + \frac{1}{T-1} \sum_{i=1}^{T-1} \|\tau_{i+1} - \tau_i\|_1 \right) \quad (15)$$

Since the magnitudes of  $\mathcal{L}_{\text{time}}$  and  $\mathcal{L}_{\text{torque}}$  may differ significantly, we introduce a dynamic scaling factor  $\beta$  to align their orders of magnitude:

$$\beta = 10^{\mathcal{O}(\mathcal{L}_{\text{time}}) - \mathcal{O}(\mathcal{L}_{\text{torque}})} \quad (16)$$

Here,  $\mathcal{O}(\cdot)$  denotes the base-10 order of magnitude. Additionally, a user-defined weight  $\gamma$  is introduced to control the relative importance of torque rate. The final loss function is defined as:

$$\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{time}} + \gamma \cdot \beta \cdot \mathcal{L}_{\text{torque}} \quad (17)$$

In our implementation, we set  $\alpha = \gamma = 1.0$  by default, and compute  $\beta$  dynamically at each iteration to ensure balanced contributions from both objectives.

To ensure physical feasibility in the joint space, velocity, acceleration, and torque constraints are incorporated into the optimization framework. The resulting multi-objective optimization function is formulated as follows:

$$\begin{aligned} \min_{\{v_i, a_i\}} \quad & \mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{time}} + \gamma \cdot \beta \cdot \mathcal{L}_{\text{torque}} \\ \text{subject to} \quad & \dot{\theta}_j^{\min} \leq \dot{\theta}_j(t) \leq \dot{\theta}_j^{\max}, \\ & \ddot{\theta}_j^{\min} \leq \ddot{\theta}_j(t) \leq \ddot{\theta}_j^{\max}, \\ & \tau_j^{\min} \leq \tau_j(t) \leq \tau_j^{\max}, \\ & \forall j \in \{1, \dots, n\}, \quad \forall t \in [0, T_{\text{total}}] \end{aligned} \quad (18)$$

Here,  $\{v_i, a_i\}$  denote the intermediate joint velocities and accelerations to be optimized. The loss terms  $\mathcal{L}_{\text{time}}$  and  $\mathcal{L}_{\text{torque}}$  are defined in Eq. (17), where  $\mathcal{L}_{\text{torque}}$  is computed from discrete torque differences normalized by the sampling interval. To solve the aforementioned multi-objective trajectory optimization problem, this work employs Particle Swarm Optimization (PSO) to perform global search over intermediate joint velocities and accelerations. PSO is a population-based metaheuristic inspired by swarm intelligence, known for its simplicity and strong exploration ability. To improve its stability and convergence performance in high-dimensional constrained spaces, the algorithm is enhanced with a constriction factor and time-varying inertia weight.

In the PSO formulation, each particle encoded a full set of velocity and acceleration values at the intermediate trajectory points. These parameters were subsequently into per-joint motion profiles and used to compute the corresponding quintic polynomial coefficients.

Let  $\mathbf{x}_k^t$  and  $\mathbf{v}_k^t$  denote the position and velocity of the  $k$ th particle at iteration  $t$ , respectively. The personal best position is stored as  $\mathbf{p}_k^t$ , and the global best position across the swarm is  $\mathbf{g}^t$ . The update rules follow the standard PSO formulation with a constriction factor  $\chi$  and time-varying inertia weight  $\omega(t)$ :

$$\mathbf{v}_k^{t+1} = \chi \cdot [\omega(t) \cdot \mathbf{v}_k^t + c_1 r_1 (\mathbf{p}_k^t - \mathbf{x}_k^t) + c_2 r_2 (\mathbf{g}^t - \mathbf{x}_k^t)] \quad (19)$$

$$\mathbf{x}_k^{t+1} = \mathbf{x}_k^t + \mathbf{v}_k^{t+1} \quad (20)$$

Here,  $c_1$  and  $c_2$  are the cognitive and social learning coefficients, while  $r_1, r_2 \sim \mathcal{U}(0, 1)$  are random variables sampled at each iteration. The constriction factor  $\chi$  is computed based on  $\varphi = c_1 + c_2$ , with  $\varphi > 4$ :

$$\chi = \frac{2}{|2 - \varphi - \sqrt{\varphi^2 - 4\varphi}|} \quad (21)$$

The inertia weight decays linearly over time:

$$\omega(t) = \omega_{\max} - (\omega_{\max} - \omega_{\min}) \cdot \frac{t}{t_{\max}} \quad (22)$$

Each particle's position vector is reshaped into per-joint velocity and acceleration sequences. These sequences help compute the full trajectory and evaluate the objective function. This encoding-decoding approach maintains interpretability and physical feasibility during optimization.

To ensure that the generated trajectories satisfy physical constraints, we formulate the fitness function by incorporating penalty terms for violations in joint velocity, acceleration, and torque:

$$f(\mathbf{x}_k^t) = \mathcal{L}_{\text{total}}(\mathbf{x}_k^t) + \lambda_1 \Psi_{\text{vel}}(\mathbf{x}_k^t) + \lambda_2 \Psi_{\text{acc}}(\mathbf{x}_k^t) + \lambda_3 \Psi_{\tau}(\mathbf{x}_k^t) \quad (23)$$

Here:

- $\mathcal{L}_{\text{total}}$  is the main loss function;

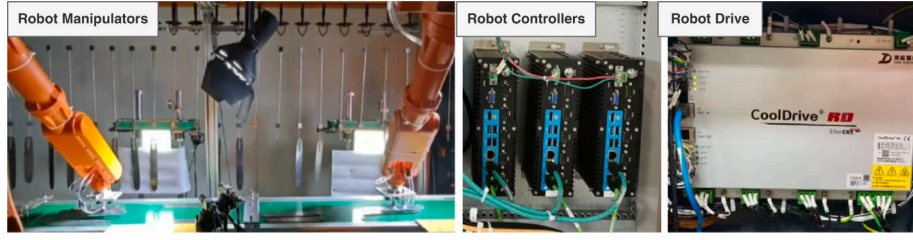


Fig. 5. Hardware configuration of the experimental robotic system, including dual-arm manipulators, corresponding motion controllers, and EtherCAT-based drive units.

- $\Psi_{vel}$ ,  $\Psi_{acc}$ , and  $\Psi_{\tau}$  are squared penalty terms measuring the degree of constraint violations for joint velocity, acceleration, and torque;
- $\lambda_1$ ,  $\lambda_2$ , and  $\lambda_3$  are user-defined penalty weights.

For improved optimization stability and feasibility under strict physical constraints, an adaptive penalty strategy is used. Specifically, the penalty weights  $\lambda_i$  are progressively increased during optimization iterations to guide particles away from infeasible regions. In addition, particles with severe violations are projected back to the nearest feasible boundary to ensure physical realizability. This strategy helps maintain convergence efficiency while preventing particles from drifting too far into invalid regions. It also enables the optimizer to explore more aggressively in the early stages and gradually focus on feasible high-quality solutions as the search progresses.

## 5. Experiment

This section presents the experimental validation of the proposed framework. Section 5.1 introduces the real industrial robotic platform used for data acquisition and validation. Section 5.2 describes the implementation environment, including hardware setup and parameter configuration. Section 5.3 compares the proposed model with other baseline methods on real-world data to evaluate prediction performance. Section 5.4 investigates model robustness under different payload conditions. Section 5.5 develops a MuJoCo-based simulation of the Qianjiang manipulator to compare simulated and real-world results, demonstrating the method's generality. Section 5.6 conducts trajectory optimization experiments to assess time–torque coordination performance. Finally, Section 5.7 provides a comprehensive discussion of the findings, highlighting the advantages, limitations, and potential extensions of the proposed approach.

### 5.1. Experimental robotic platform

The hardware configuration of the experimental robotic system is shown in Fig. 5. The setup comprises two six-axis industrial manipulators, corresponding motion controllers, and an EtherCAT-based drive unit. Each manipulator provides three translational and three rotational degrees of freedom, allowing flexible motion planning for complex pick-and-place and alignment tasks. The manipulators are equipped with dual-suction grippers mounted on standardized flanges, enabling simultaneous handling of two workpieces during operation. All motion commands and sensor feedback are coordinated through EtherCAT communication, ensuring synchronized control between the dual manipulators. The controllers execute joint-level commands in real time, while the drive unit supplies precise torque and velocity control for each actuator.

Each manipulator in the experimental platform is a Qianjiang QJR15-1500, a six-degree-of-freedom industrial robot configured for the 6 kg rated-load version of the series. As summarized in Table 1, the robot provides a wide range of joint motion together with high rotational speed and acceleration limits, enabling precise evaluation of dynamic performance. A vacuum gripper weighing approximately 1 kg

is mounted on the end flange, allowing simultaneous manipulation of two workpieces.

The workpieces used in the experiments vary in size from 6 inches to 18 inches, with individual masses ranging between 0.2 kg and 1 kg, depending on the specification. For consistency and reproducibility, all experiments are conducted using standard 12-inch workpieces with a nominal mass of 0.5 kg each. The manipulators communicate via an EtherCAT fieldbus with a control cycle of 1 ms, following the PLCopen motion-control standard to ensure deterministic and synchronized execution of joint commands. The combination of variable payloads and high operation speeds introduces considerable dynamic variations, providing a representative and challenging scenario for validating the proposed torque-aware trajectory optimization framework.

### 5.2. Implementation and experimental configuration

Building upon the described robotic platform, the implementation and parameter settings of the proposed framework are detailed as follows. All learning-based experiments were implemented in PyTorch and executed on a workstation running Windows 11, equipped with an Intel Core i9-13900K processor, 32 GB of RAM, and an NVIDIA RTX 4090 GPU with 24 GB of VRAM.

The torque prediction model was trained using joint motion data collected from a pre-deployed robotic platform, including joint positions, velocities, and torques. Each data sample contained 18 features—six joint positions, six joint velocities, and six torque values. The lite and full model variants used input dimensions of 12 and 18, respectively, corresponding to whether torque history was included. The model output was the predicted torque vector of six joints at the next time step. The models were trained for 200 epochs using the Adam optimizer with a learning rate of 0.001 and a batch size of 1024. The dataset was randomly divided into 80% training and 20% validation subsets.

For the trajectory optimization, the Particle Swarm Optimization (PSO) algorithm was employed to minimize the total execution time and torque variation. The swarm size was set to 80, with a maximum of 50 iterations. The inertia weight was configured as  $w = 0.6$ , and the cognitive and social coefficients were  $c_1 = 2.5$  and  $c_2 = 1.0$ , respectively. The predicted torques from the neural model were integrated into the cost function to guide the swarm toward dynamically feasible trajectories with smoother torque evolution.

### 5.3. Torque prediction comparison experiments

Six sets of point-to-point motion data were collected from the robotic platform to evaluate the proposed torque prediction model. During motion execution, joint position, velocity, and torque signals were acquired through the servo drive system. The position and velocity data were converted into physical quantities using calibrated motor and gear parameters, while the torque signals were directly recorded in units of thousandths of the rated torque according to the robot drive configuration, without further conversion to absolute physical values. Each trajectory point is represented in the Tool Center Point (TCP) coordinate frame as a six-dimensional vector  $(x, y, z, R_x, R_y, R_z)$ .

**Table 1**

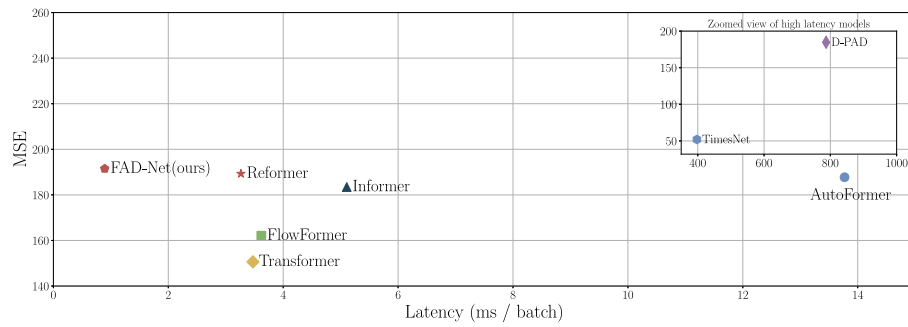
Joint-level specifications of the 6-DOF QJR15-1500 industrial robot configured for the 6 kg maximum payload.

|                             | Joint 1 | Joint 2  | Joint 3  | Joint 4 | Joint 5 | Joint 6 |
|-----------------------------|---------|----------|----------|---------|---------|---------|
| Rated Torque (Nm)           | 7.16    | 9.60     | 2.39     | 1.27    | 0.64    | 0.64    |
| Gear Ratio                  | 36 409  | 55 637   | 55 721   | 23 763  | 22 695  | 17 920  |
| Joint Limits (°)            | ±168    | +159/−93 | +92/−125 | ±185    | ±115    | ±360    |
| Max Vel (°/s)               | 234     | 234      | 240      | 372     | 630     | 687     |
| Max Acc (°/s <sup>2</sup> ) | 916     | 744      | 1375     | 2864    | 3437    | 4583    |

**Table 2**

Start and end poses of the training trajectories, along with the number of sampled points collected during motion execution.

| Trajectory (Points) | Start Pose |      |     |                |                |                | End Pose |      |     |                |                |                |
|---------------------|------------|------|-----|----------------|----------------|----------------|----------|------|-----|----------------|----------------|----------------|
|                     | X          | Y    | Z   | R <sub>x</sub> | R <sub>y</sub> | R <sub>z</sub> | X        | Y    | Z   | R <sub>x</sub> | R <sub>y</sub> | R <sub>z</sub> |
| T1 (2058)           | 1000       | −300 | 700 | 179            | 0              | 0              | 1200     | 300  | 950 | 170            | −90            | 10             |
| T2 (1369)           | 781        | 450  | 600 | 179            | 0              | 90             | 1300     | −400 | 800 | 160            | −17            | 52             |
| T3 (1710)           | 501        | 350  | 601 | 179            | 0              | 90             | 1300     | −400 | 800 | 160            | −17            | 52             |
| T4 (3110)           | 1300       | −500 | 900 | −90            | −80            | −90            | 500      | 350  | 600 | 179            | 0              | 90             |
| T5 (1976)           | 500        | 350  | 600 | 179            | 0              | 90             | 1300     | −400 | 700 | 179            | 0              | 0              |



**Fig. 6.** Comparison of mean squared error (MSE) and average inference latency among different torque prediction models under their respective optimal timesteps. Each point represents the average performance over the test set with a batch size of 1024. The inset highlights models with high-latency characteristics for clearer visualization.

The intermediate points were used to construct sequential samples for supervised training and evaluation. As summarized in Table 2, the dataset covers diverse path lengths, directions, and dynamic characteristics, thereby supporting comprehensive generalization analysis of the proposed model.

We first evaluated the proposed model alongside several representative time-series forecasting architectures, including five Transformer-based models, Autoformer [28], Flowformer [29], Informer [30], the standard Transformer [27], and Reformer [31], as well as two non-Transformer baselines, D-PAD [47] with delay-aware projection and TimesNet [48] with multi-scale convolutional blocks. For each model, the most suitable timestep configuration was determined through preliminary analysis to achieve optimal performance within its own architecture. All models were trained and tested under identical data partitions and hyperparameter settings. The evaluation focused on the mean squared error (MSE) of predicted torques and the average inference latency per batch, reflecting both predictive accuracy and computational efficiency.

As shown in Fig. 6, Transformer-based models generally achieve competitive accuracy but exhibit considerably higher inference latency. FlowFormer and the standard Transformer produce MSE values about 10%–20% lower than that of the proposed model, yet their average inference time per batch is roughly three times longer—3.6 ms and 3.5 ms per batch, compared to 0.89 ms for the proposed model. In contrast, TimesNet attains the lowest MSE among all models but requires more than two orders of magnitude longer inference time, over 300 times slower than the proposed model. These results indicate that the proposed model achieves a more favorable balance between predictive accuracy and computational efficiency, offering clear advantages for latency-sensitive industrial applications.

To further investigate the influence of temporal attention mechanisms on torque prediction, we conducted independent experiments using representative attention kernels from Transformer-based architectures. In this comparison, the attention modules of Flowformer, the standard Transformer, and Reformer were implemented under identical input, training, and evaluation settings. These kernels were selected for their balance between computational efficiency and modeling capability, allowing us to assess how different attention formulations affect predictive accuracy and inference latency.

As shown in Table 3, in the lite mode, FAD-Net achieves the best prediction accuracy among all models, reaching an MSE of 191.5 at the optimal timestep. Although it outperforms all models except FlowAttention, its advantage over FlowAttention is relatively limited—the MSE decreases only from 195.7 to 191.5, while the inference latency nearly doubles. In contrast, in the full mode, FAD-Net exhibits a much more pronounced superiority, achieving significantly lower MSE than all other attention mechanisms while maintaining a consistently low inference latency. Moreover, the full mode also outperforms the lite mode by a considerable margin, demonstrating that incorporating torque feedback substantially enhances the model's ability to capture dynamic dependencies and predict joint torque variations more accurately.

In summary, the comparative experiments demonstrate that FAD-Net consistently achieves superior performance in both lite and full prediction modes. While the lite mode offers a good trade-off between accuracy and speed, the full mode further enhances predictive precision by leveraging torque feedback to capture dynamic dependencies more effectively. Compared with existing attention-based models, FAD-Net maintains competitive accuracy while achieving markedly lower inference latency, indicating its suitability for real-time torque prediction and control applications. Nevertheless, the model still exhibits certain limitations — its inference speed remains slower than the



**Table 3**

Comparison of torque prediction accuracy (MSE, MAE) and inference latency (ms per batch, batch size = 1024) across different attention mechanisms under full and lite modes.

| Lite Mode Prediction |                |       |         |               |       |         |               |       |         |               |       |         |
|----------------------|----------------|-------|---------|---------------|-------|---------|---------------|-------|---------|---------------|-------|---------|
| Time steps           | FAD-Net (ours) |       |         | FlowAttention |       |         | FullAttention |       |         | ReformerLayer |       |         |
|                      | MSE            | MAE   | Latency | MSE           | MAE   | Latency | MSE           | MAE   | Latency | MSE           | MAE   | Latency |
| 16                   | 315.6          | 13.20 | 0.57    | <b>201.6</b>  | 10.47 | 0.64    | 298.8         | 12.91 | 0.22    | 338.0         | 13.31 | 1.19    |
| 32                   | 292.2          | 12.16 | 0.70    | <b>195.0</b>  | 10.34 | 0.66    | 259.4         | 12.12 | 0.67    | 310.5         | 12.79 | 1.93    |
| 64                   | 201.5          | 10.46 | 0.89    | <b>198.2</b>  | 10.31 | 0.80    | 230.1         | 11.58 | 1.42    | 274.4         | 12.12 | 5.02    |
| 128                  | <b>191.5</b>   | 10.17 | 1.37    | 205.0         | 10.57 | 1.32    | 192.1         | 10.17 | 4.53    | 204.6         | 10.48 | 12.30   |
| 256                  | <b>195.7</b>   | 10.24 | 2.78    | 207.6         | 10.80 | 3.01    | 202.8         | 10.26 | 16.14   | 199.2         | 10.28 | 29.08   |

| Full Mode Prediction |                |      |         |               |      |         |               |      |         |               |      |         |
|----------------------|----------------|------|---------|---------------|------|---------|---------------|------|---------|---------------|------|---------|
| Time steps           | FAD-Net (ours) |      |         | FlowAttention |      |         | FullAttention |      |         | ReformerLayer |      |         |
|                      | MSE            | MAE  | Latency | MSE           | MAE  | Latency | MSE           | MAE  | Latency | MSE           | MAE  | Latency |
| 2                    | <b>33.2</b>    | 3.95 | 0.48    | 34.1          | 4.02 | 0.64    | 66.1          | 5.90 | 0.25    | 57.7          | 5.27 | 1.52    |
| 4                    | <b>29.4</b>    | 3.62 | 0.46    | 36.2          | 4.25 | 0.66    | 62.5          | 5.54 | 0.22    | 63.6          | 5.30 | 1.45    |
| 8                    | 58.6           | 5.27 | 0.53    | <b>46.2</b>   | 4.91 | 0.65    | 73.2          | 6.02 | 0.21    | 66.1          | 5.38 | 1.17    |
| 16                   | 101.2          | 6.42 | 0.53    | 72.8          | 6.12 | 0.65    | 94.4          | 6.77 | 0.24    | 70.4          | 6.18 | 1.21    |
| 32                   | 84.4           | 5.79 | 0.65    | 94.6          | 7.20 | 0.67    | 106.7         | 7.59 | 0.74    | <b>64.5</b>   | 5.73 | 2.25    |

**Table 4**

Task point definitions for the grasping and hanging operations. Dashed lines indicate a 50 ms waiting time between trajectory segments. All poses are represented in TCP coordinates ( $x, y, z, R_x, R_y, R_z$ ).

| Grasping Task |         |        |       |       |       |       |       |
|---------------|---------|--------|-------|-------|-------|-------|-------|
| Segment       | Step    | $x$    | $y$   | $z$   | $R_x$ | $R_y$ | $R_z$ |
| Grasp 1       | Start   | 1250.0 | -90.0 | 600.0 | 160.0 | -17   | 52.0  |
|               | Above 1 | 505.9  | 274.7 | 441.0 | 179.9 | 0     | 91.7  |
|               | Grasp 1 | 505.9  | 274.7 | 425.0 | 179.9 | 0     | 91.7  |
| Grasp 2       | Lift    | 505.9  | 274.7 | 441.0 | 179.9 | 0     | 91.7  |
|               | Above 2 | 510.3  | 277.2 | 441.0 | 179.9 | 0     | 90.9  |
|               | Grasp 2 | 510.3  | 277.2 | 415.0 | 179.9 | 0     | 90.9  |
| Hang 1        | Lift    | 510.3  | 277.2 | 441.0 | 179.9 | 0     | 90.9  |

| Hanging Task |         |        |        |       |       |       |       |
|--------------|---------|--------|--------|-------|-------|-------|-------|
| Segment      | Step    | $x$    | $y$    | $z$   | $r_x$ | $r_y$ | $r_z$ |
| Hang 1       | Start   | 1250.0 | -90.0  | 600.0 | 160.0 | -17   | 52.0  |
|              | Hang 1  | 1315.0 | -167.2 | 598.5 | 160.0 | -17   | 52.0  |
| Hang 2       | Retreat | 1345.0 | -107.2 | 648.5 | 160.0 | -17   | 52.0  |
|              | Hang 2  | 1361.6 | -147.3 | 631.9 | 160.0 | -17   | 52.0  |

most lightweight attention variants such as FlowAttention, and the performance gain in the lite mode is relatively limited when torque feedback is unavailable.

#### 5.4. Payload variation experiment

To further evaluate the robustness of the proposed model under different dynamic conditions, additional experiments were conducted with varying end-effector loads ranging from 1 kg to 5 kg. During each trial, the robot executed a standardized manipulation routine consisting of grasping and hanging operations, as summarized in Table 4. The task comprises two main stages — the Grasping Task and the Hanging Task — which are further divided into four continuous motion segments: Grasp 1, Grasp 2, Hang 1, and Hang 2. Each segment begins precisely at the terminal pose of the preceding one, forming a seamless motion chain throughout the entire sequence. To ensure stable execution under all payload conditions, the end-effector remains stationary for at least 50 ms at each key manipulation point, including both grasping and releasing events.

As shown in Fig. 7, the mean absolute error (MAE) of the lite mode is employed as the evaluation metric to quantify the prediction deviation for each joint, as it directly measures the magnitude of error without amplifying it through squaring. The results indicate that Joints 2, 3, and 5 exhibit relatively higher MAE values, corresponding to their roles as the primary torque-bearing axes during the manipulation process. With increasing payload from 1 kg to 5 kg, the MAE of Joints

3 and 5 shows a slight upward trend, reflecting mild sensitivity to load variations. Compared with the previous single-trajectory evaluation, the overall MAE values in this experiment are slightly higher. This is mainly because the dataset used here consists of multiple concatenated motion segments, where transitional discontinuities between segments weaken the temporal smoothness of the input sequence and thereby reduce prediction precision. Nevertheless, the overall accuracy remains stable across all joints, demonstrating that the proposed model maintains strong robustness and generalization capability under varying payloads and motion continuity conditions.

#### 5.5. Simulation-experimental comparison

Beyond the real-world tests, a dynamic simulation model of the Qianjiang robotic manipulator was further developed in MuJoCo to examine the universality of the proposed method under controlled environments. The torque prediction performance was evaluated under two payload conditions — light load (0 kg) and heavy load (5 kg) — for both the Lite and Full modes. To ensure consistent evaluation, the torque data collected from the physical robot — originally expressed as per mille of the rated torque — were converted into physical units (N m), consistent with the outputs of the MuJoCo inverse dynamics engine.

The final comparison results are presented in Fig. 8. The proposed model achieves high prediction accuracy under both real-world and simulated conditions, demonstrating its robustness and generalization

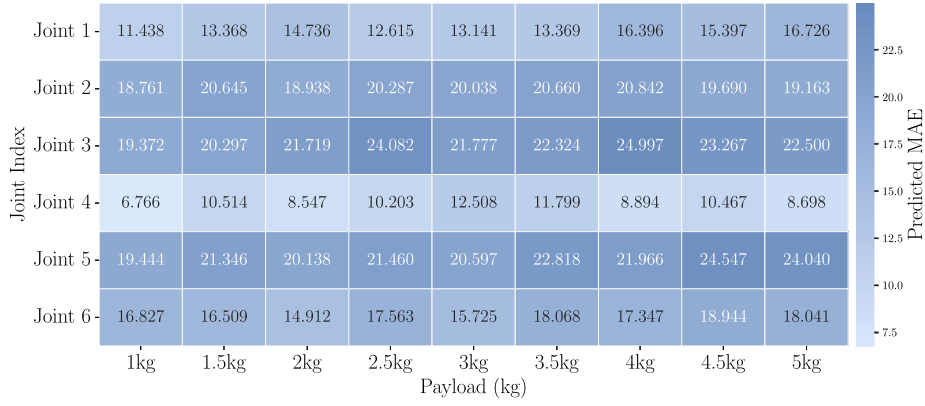


Fig. 7. Predicted mean absolute error (MAE) of joint torque under different payloads.

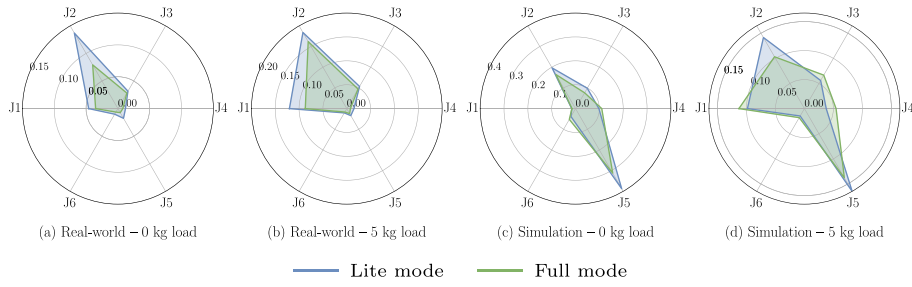


Fig. 8. Radar plots of joint-wise torque prediction errors under different environments and payloads (unit: N m). Each radar chart displays the prediction error for the six joints in both Lite and Full modes.

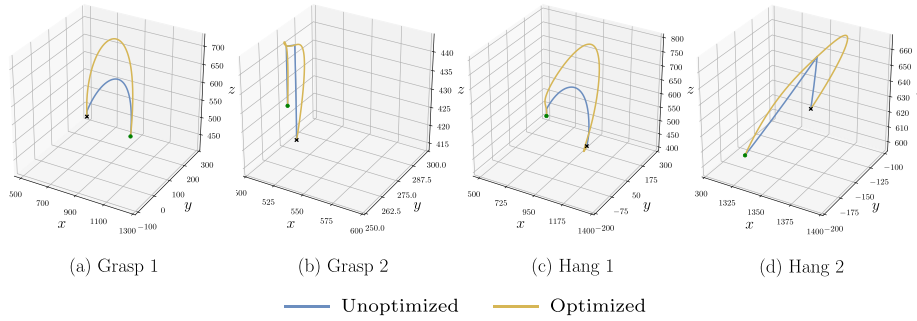


Fig. 9. Comparison of TCP trajectories before and after optimization. Circles denote the start points, and crosses denote the end points of each trajectory.

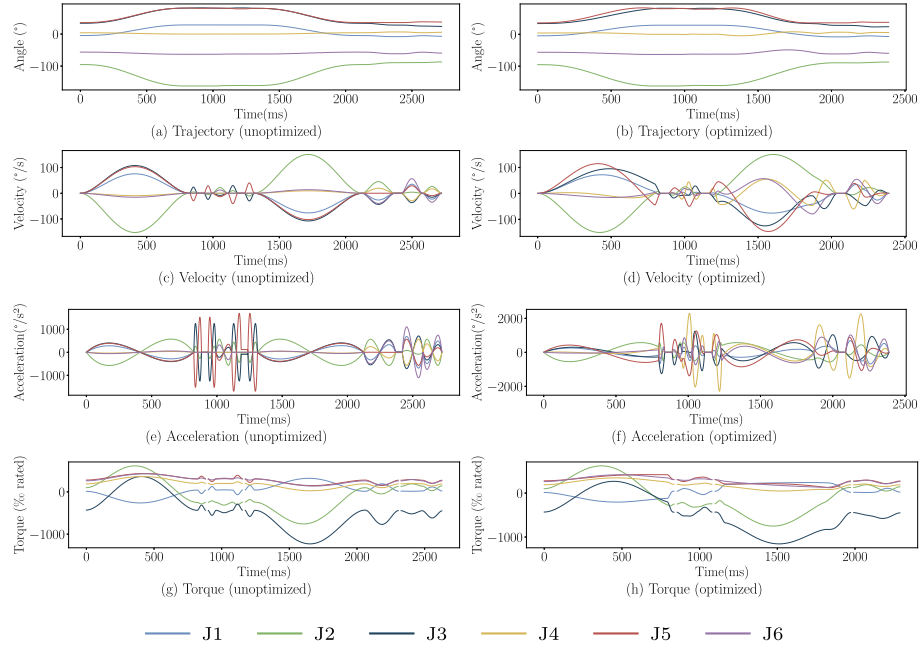
capability across different domains. It can be observed that in simulation, the performance gap between the Lite and Full modes is negligible, whereas in the real-world experiments, the Lite mode performs noticeably worse than the Full mode. This indicates that torque feedback plays a more critical role in real robotic systems, where unmodeled dynamics, frictional disturbances, and sensor noise introduce additional uncertainties that the Lite mode cannot fully compensate for.

### 5.6. Trajectory optimization experiments

The trajectory optimization experiments were conducted on the same robotic manipulation task described in Table 4. The optimization process aimed to minimize both execution time and torque fluctuation while maintaining kinematic feasibility and smooth motion transitions. As illustrated in Fig. 9, the optimized trajectory exhibits enhanced smoothness and continuity, particularly in segments where the original path contained abrupt directional changes. The refined trajectory effectively mitigates rigid cornering behavior and improves global motion coherence, leading to more stable joint actuation and reduced mechanical stress during task execution.

To further verify the dynamic benefits of the optimized trajectory, Fig. 10 compares the joint-level position, velocity, acceleration, and predicted torque profiles between the unoptimized (left column) and optimized (right column) conditions. The optimized trajectories reveal a noticeable reduction in torque peaks across several joints, demonstrating that the proposed optimization framework effectively mitigates excessive dynamic loads and suppresses abrupt torque fluctuations. This improvement contributes to smoother torque evolution and enhanced motion stability during the execution process.

As shown in Table 5, the proposed trajectory optimization method effectively reduces both execution time and torque rate across all motion segments. The most significant improvement in execution time occurs in the first hanging phase, with a 22.99% reduction, followed by the second grasping phase (11.76%). In terms of torque rate, the greatest reduction is observed in the second grasping phase (25.76%), indicating smoother torque transitions. Such improvements stem from the adaptive adjustment of joint velocities and accelerations at intermediate trajectory points, which eliminates unnecessary stops and enhances dynamic stability. Even in shorter or already optimized segments, modest gains are achieved, confirming the robustness of the



**Fig. 10.** Comparison of joint motion states under unoptimized (left) and optimized (right) trajectories: (a-b) position, (c-d) velocity, (e-f) acceleration, and (g-h) predicted torque. To facilitate comparison across trajectory segments, torque predictions are separated by blank intervals between motion phases.

**Table 5**

Execution time and torque rate before and after optimization. U: Unoptimized, O: Optimized,  $\Delta T$ : Time saved,  $\Delta R$ : Rate reduction (%).

|                | Grasp 1      | Grasp 2      | Hang 1       | Hang 2       |
|----------------|--------------|--------------|--------------|--------------|
| Time (U)       | 0.8815       | 0.2524       | 1.1320       | 0.3029       |
| Time (O)       | 0.8366       | 0.2227       | 0.8717       | 0.2986       |
| $\Delta T$ (%) | <b>5.09</b>  | <b>11.76</b> | <b>22.99</b> | <b>1.42</b>  |
| Rate (U)       | 978.49       | 1269.32      | 1082.69      | 840.51       |
| Rate (O)       | 830.94       | 942.40       | 830.29       | 668.84       |
| $\Delta R$ (%) | <b>15.06</b> | <b>25.76</b> | <b>23.32</b> | <b>20.40</b> |

proposed strategy. Overall, the optimization enhances both temporal efficiency and motion smoothness, contributing to more stable and efficient robotic execution.

Building on the improvements in execution time and torque rate, further analysis was performed from both time- and frequency-domain perspectives. As shown in Fig. 11, four motion metrics were compared before and after optimization: maximum jerk, average jerk, high-frequency ratio, and spectral entropy. After optimization, most joints show clear reductions in maximum and average jerk, especially J3 and J5, indicating smoother acceleration transitions. Spectral entropy also decreases, suggesting lower frequency complexity and more stable dynamics. The high-frequency ratio remains steady, confirming that the optimization suppresses torque peaks without introducing additional noise. Overall, the method improves both motion smoothness and dynamic regularity.

To evaluate the convergence speed and stability of different heuristic algorithms, we compare three representative methods: CMA-ES, DE, and PSO. As shown in Fig. 12, both DE and PSO exhibit rapid convergence and maintain stable, low cost values thereafter. In contrast, CMA-ES shows significant fluctuations and slower convergence across all tasks. Overall, PSO demonstrates the fastest and most stable convergence among the three, and thanks to its lower computational complexity and fewer hyperparameters, it is more suitable for industrial applications with strict real-time requirements.

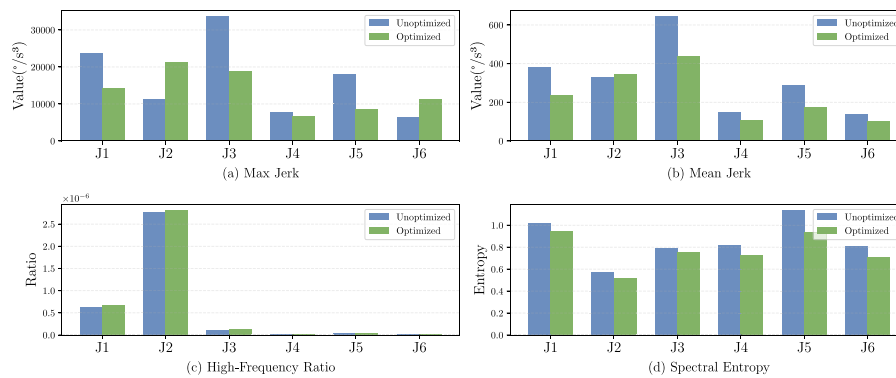
## 5.7. Discussion

The proposed torque-aware optimization framework achieves a good balance between prediction accuracy, inference efficiency, and robustness under different motion and load conditions. The dual-mode structure allows flexible deployment across various robotic platforms: the Lite mode provides high computational efficiency, while the Full mode benefits from torque feedback to enhance dynamic awareness and accuracy. Since the framework relies only on basic joint state information, it can be readily applied to different manipulators equipped with standard joint sensing interfaces. The simplified attention mechanism enables efficient temporal modeling for real-time inference, and the PSO-based trajectory optimizer effectively suppresses torque oscillations while maintaining improved time efficiency.

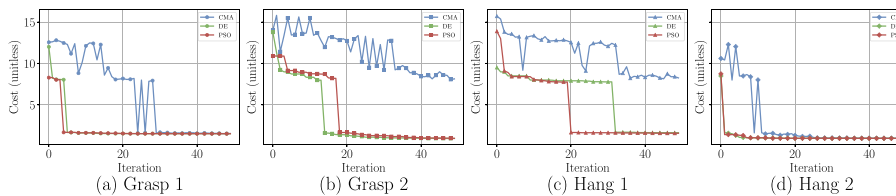
However, several limitations remain. The prediction accuracy decreases slightly when motion segments with different dynamic properties are concatenated, reflecting sensitivity to temporal discontinuities. Moreover, the heuristic optimization must be repeatedly solved for each new trajectory and cannot learn from prior optimization experiences, limiting its adaptability and real-time scalability. These issues should be carefully considered when extending the method to more complex or continuously changing industrial tasks.

## 6. Conclusion

This paper presents a torque-aware trajectory planning framework that integrates delay-sensitive torque prediction with heuristic joint optimization to generate physically feasible and time-efficient robotic motions. The proposed model supports both full (with torque feedback) and lite (motion-only) modes, enabling flexible deployment across different robotic platforms. Experimental evaluations on a six-axis industrial robot demonstrate consistent improvements in execution efficiency and torque smoothness. Owing to its modular design and low sensing requirements, the framework can be readily integrated into existing robotic control systems in manufacturing environments. It is particularly applicable to torque-constrained and precision-sensitive tasks such as assembly, polishing, and load-handling operations, where stable torque transitions and high motion coordination are essential for maintaining product quality and system reliability.



**Fig. 11.** Bar chart comparison of four motion quality metrics — maximum jerk, average jerk, high-frequency ratio, and spectral entropy — across all joints before and after trajectory optimization.



**Fig. 12.** Convergence comparison of CMA-ES, DE, and PSO.

Future work will aim to further improve the model's prediction accuracy and computational efficiency under complex dynamic conditions. Particular attention will be devoted to enhancing torque prediction performance at motion transitions, where dynamic discontinuities and coupling effects are most pronounced. In addition, the current point-wise state optimization will be extended toward global trajectory-level optimization to achieve smoother and more coordinated motion profiles. Future research will also explore adaptive online adjustment using real-time sensor feedback and investigate lightweight deployment on embedded platforms to enable low-latency execution in practical industrial environments.

#### CRediT authorship contribution statement

**Zeyun Xiao:** Writing – review & editing, Writing – original draft, Validation, Investigation, Formal analysis, Conceptualization. **Danfeng Sun:** Writing – review & editing, Writing – original draft, Methodology, Investigation. **Donglai Zhu:** Visualization, Validation. **Yong Wang:** Writing – review & editing, Validation. **Yi Yan:** Methodology, Conceptualization. **Huifeng Wu:** Writing – review & editing, Methodology, Funding acquisition, Conceptualization.

#### Funding

This work was supported in part by a grant from the National Natural Science Foundation of China (No. U21A20484 and No. 62303145).

#### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

#### Data availability

Data will be made available on request.

#### References

- [1] S. Huang, M. Wang, Y. Zou, V-rep-based virtual platform on multi-pattern neural control of robotic manipulator, in: 2019 Chinese Control and Decision Conference, CCDC, IEEE, 2019, pp. 1570–1575, <http://dx.doi.org/10.1109/ccdc.2019.8832664>.
- [2] M. Wu, A. Rupenyan, B. Corves, Autogeneration and optimization of pick-and-place trajectories in robotic systems: A data-driven approach, *Robot. Comput.-Integr. Manuf.* 97 (2026) 103080, <http://dx.doi.org/10.1016/j.rcim.2025.103080>.
- [3] T. Kot, Z. Bobovsky, A. Vysocký, V. Kryš, J. Šafařík, R. Ružarovský, Method for robot manipulator joint wear reduction by finding the optimal robot placement in a robotic cell, *Appl. Sci.* 11 (12) (2021) 5398, <http://dx.doi.org/10.3390/app11125398>.
- [4] C. Ji, Z. Zhang, G. Cheng, M. Kong, R. Li, A convex optimization method to time-optimal trajectory planning with jerk constraint for industrial robotic manipulators, *IEEE Trans. Autom. Sci. Eng.* 21 (4) (2023) 7629–7646, <http://dx.doi.org/10.1109/tase.2023.3346693/mml>.
- [5] G.R. Nazmara, P. Neto, A.P. Aguiar, Robust learning-based impedance control of robotic manipulators, in: *CONTROLO 2024: Proceedings of the 16th APCA International Conference on Automatic Control and Soft Computing*, Vol. 1325, July (2024) 17–19, Porto, Portugal, Springer Nature, 2025, p. 297, [http://dx.doi.org/10.1007/978-3-031-81724-3\\_27](http://dx.doi.org/10.1007/978-3-031-81724-3_27).
- [6] P. Yue, B. Xu, M. Zhang, An improve nonlinear robust control approach for robotic manipulators with pso-based global optimization strategy, *Sci. Rep.* 14 (1) (2024) 21447, <http://dx.doi.org/10.1038/s41598-024-72156-x>.
- [7] A. Ghediri, K. Lamamra, A.A. Kaki, S. Vaidyanathan, Adaptive pid computed-torque control of robot manipulators based on ddpq reinforcement learning, *Int. J. Model. Identif. Control.* 41 (3) (2022) 173–182, <http://dx.doi.org/10.1504/IJMIC.2022.127518>.
- [8] S. Kružić, J. Musić, R. Kamnik, V. Papić, End-effector force and joint torque estimation of a 7-dof robotic manipulator using deep learning, *Electronics* 10 (23) (2021) 2963, <http://dx.doi.org/10.3390/electronics10232963>.
- [9] Z. Huo, M. Yuan, S. Zhang, X. Zhang, Observer-based adaptive robust force control of a robotic manipulator integrated with external force/torque sensor, *Actuators* 14 (3) (2025) <http://dx.doi.org/10.3390/act14030116>.
- [10] Z. Liu, R. Li, L. Zhao, Y. Xia, Z. Qin, K. Zhu, Automatic joint motion planning of 9-dof robot based on redundancy optimization for wheel hub polishing, *Robot. Comput.-Integr. Manuf.* 81 (2023) 102500, <http://dx.doi.org/10.1016/j.rcim.2022.102500>.
- [11] Z. Lončarević, A. Gams, S. Reberšek, B. Nemec, J. Škrabar, J. Skvarč, A. Ude, Specifying and optimizing robotic motion for visual quality inspection, *Robot. Comput.-Integr. Manuf.* 72 (2021) 102200, <http://dx.doi.org/10.1016/j.rcim.2021.102200>.



- [12] H. Wu, J. Yang, S. Huang, X. Ning, Z. Zhang, Multi-objective adaptive trajectory optimization for industrial robot based on acceleration continuity constraint, *Robot. Comput.-Integr. Manuf.* 84 (2023) 102597, <http://dx.doi.org/10.1016/j.rcim.2023.102597>.
- [13] R. De Laet, N. Van Oosterwyck, L. Scalera, A. Cuyt, A. Gasparetto, S. Deramme-laere, Energy-efficient motion planning for robotic systems using polynomials in the chebyshev basis, *Robot. Auton. Syst.* (2025) 105051, <http://dx.doi.org/10.1016/j.robot.2025.105051>.
- [14] C. Garriz, R. Domingo, Trajectory optimization in terms of energy and performance of an industrial robot in the manufacturing industry, *Sensors* 22 (19) (2022) 7538, <http://dx.doi.org/10.3390/s22197538>.
- [15] S. Zhu, Y. Zhang, Z. Li, J. Wang, S. Yuan, Multi-objective optimal trajectory planning method for robot manipulator with real-time parameters monitoring, *Mech. Syst. Signal Process.* 229 (2025) 112518, <http://dx.doi.org/10.1016/j.ymssp.2025.112518>.
- [16] B. Chen, Y. Wang, S. Hu, Z. Tao, J. Qi, A whole-path posture optimization method of robotic grinding based on multi-performance evaluation indices, *Robot. Comput.-Integr. Manuf.* 89 (2024) 102787, <http://dx.doi.org/10.1016/j.rcim.2024.102787>.
- [17] Y. Zhang, Z. Sun, Q. Sun, Y. Wang, Time-energy consumption optimal path-constrained trajectory planning of excavator robotics, *J. Eng.* 2024 (6) (2024) e12406, <http://dx.doi.org/10.1049/tje2.12406>.
- [18] G. Fabris, L. Scalera, A. Gasparetto, Online optimization of minimum-time and minimum-energy trajectories for a 1-dof belt-driven robotic system, *Robotica* (2025) 1–16, <http://dx.doi.org/10.1017/S0263574725001456>.
- [19] F. Lozer, L. Scalera, P. Boscariol, A. Gasparetto, Planning optimal minimum-jerk trajectories for redundant robots, *Robot. Auton. Syst.* (2025) 105049, <http://dx.doi.org/10.1016/j.robot.2025.105049>.
- [20] N. Bendali, L. Scalera, A. Sundaram, A. Said, A. Gasparetto, An approach based on multiquadric radial basis functions for smooth trajectory planning of robotic manipulators with kinematic constraints, *Nonlinear Dynam.* (2025) 1–19, <http://dx.doi.org/10.1007/s11071-025-11248-3>.
- [21] S.S.W. Fatima, A. Rahimi, A review of time-series forecasting algorithms for industrial manufacturing systems, *Machines* 12 (6) (2024) 380, <http://dx.doi.org/10.3390/machines12060380>.
- [22] G.E. Box, G.M. Jenkins, G.C. Reinsel, G.M. Ljung, *Time Series Analysis: Forecasting and Control*, John Wiley & Sons, 2015, URL <https://www.wiley.com/en-us/Time+Series+Analysis%3A+Forecasting+and+Control%2C+5th+Edition-p-9781118675021>.
- [23] R.E. Kalman, A new approach to linear filtering and prediction problems, *J. Basic Eng.* 82 (1) (1960) 35–45, <http://dx.doi.org/10.1115/1.3662552>.
- [24] A.J. Smola, B. Schölkopf, A tutorial on support vector regression, *Stat. Comput.* 14 (2004) 199–222, <http://dx.doi.org/10.1023/B:STCO.0000035301.49549.88>.
- [25] A. Wahid, J.G. Breslin, M.A. Intizar, Prediction of machine failure in industry 4.0: a hybrid cnn-lstm framework, *Appl. Sci.* 12 (9) (2022) 4221, <http://dx.doi.org/10.3390/app12094221>.
- [26] J. Li, B. Yang, H. Li, Y. Wang, C. Qi, Y. Liu, Dtdr-alstm: Extracting dynamic time-delays to reconstruct multivariate data for improving attention-based lstm industrial time series prediction models, *Knowl.-Based Syst.* 211 (2021) 106508, <http://dx.doi.org/10.1016/j.knsys.2020.106508>.
- [27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. u. Kaiser, I. Polosukhin, Attention is all you need, in: I. Guyon, U.V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett (Eds.), in: *Advances in Neural Information Processing Systems*, vol. 30, Curran Associates, Inc., 2017, URL [https://proceedings.neurips.cc/paper\\_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf).
- [28] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting, in: M. Ranzato and a. Beygelzimer and Y. Dauphin and P. Liang and J.W. Vaughan, in: *Advances in Neural Information Processing Systems*, vol. 34, Curran Associates, Inc., 2021, pp. 22419–22430, URL [https://proceedings.neurips.cc/paper\\_files/paper/2021/file/bcc0d400288793e8bdcd7c19a8ac0c2b-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2021/file/bcc0d400288793e8bdcd7c19a8ac0c2b-Paper.pdf).
- [29] Z. Huang, X. Shi, C. Zhang, Q. Wang, K.C. Cheung, H. Qin, J. Dai, H. Li, Flowformer: A transformer architecture for optical flow, in: *European Conference on Computer Vision*, Springer, 2022, pp. 668–685, [http://dx.doi.org/10.1007/978-3-031-19790-1\\_40](http://dx.doi.org/10.1007/978-3-031-19790-1_40).
- [30] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, W. Zhang, Informer: Beyond efficient transformer for long sequence time-series forecasting, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, (12) 2021, pp. 11106–11115, <http://dx.doi.org/10.1609/aaai.v35i12.17325>.
- [31] N. Kitaev, L. Kaiser, A. Levskaya, Reformer: The efficient transformer, 2020, <http://dx.doi.org/10.48550/arXiv.2001.04451>, arXiv preprint [arXiv:2001.04451](https://arxiv.org/abs/2001.04451).
- [32] C. Wang, H. Wang, X. Zhang, Q. Liu, M. Liu, G. Xu, A transformer-based industrial time series prediction model with multivariate dynamic embedding, *IEEE Trans. Ind. Informat.* (2024) <http://dx.doi.org/10.1109/TII.2024.3488783>.
- [33] Y. Jin, L. Hou, Y. Chen, A time series transformer based method for the rotating machinery fault diagnosis, *Neurocomputing* 494 (2022) 379–395, <http://dx.doi.org/10.1016/j.neucom.2022.04.111>.
- [34] I.A. Sucan, M. Moll, L.E. Kavraki, The open motion planning library, *IEEE Robot. Autom. Mag.* 19 (4) (2012) 72–82, <http://dx.doi.org/10.1109/mra.2012.2205651>.
- [35] S.M. LaValle, J.J. Kuffner, Rapidly-exploring random trees: Progress and prospects: Steven M. Lavalle, Iowa State University, a James J. Kuffner, Jr. University of Tokyo, Tokyo, Japan, *Algorithmic Comput. Robot.* (2001) 303–307, <http://dx.doi.org/10.1201/9781439864135-43>.
- [36] L.E. Kavraki, P. Svestka, J.-C. Latombe, M.H. Overmars, Probabilistic roadmaps for path planning in high-dimensional configuration spaces, *IEEE Trans. Robot. Autom.* 12 (4) (1996) 566–580, <http://dx.doi.org/10.1109/70.508439>.
- [37] S. Karaman, E. Frazzoli, Sampling-based algorithms for optimal motion planning, *Int. J. Robot. Res.* 30 (7) (2011) 846–894, <http://dx.doi.org/10.1177/0278364911406761>.
- [38] X. Yu, M. Dong, W. Yin, Time-optimal trajectory planning of manipulator with simultaneously searching the optimal path, *Comput. Commun.* 181 (2022) 446–453, <http://dx.doi.org/10.1016/j.comcom.2021.10.005>.
- [39] H. Kim, B.K. Kim, Energy-optimal transport trajectory planning and online trajectory modification for holonomic robots, *Asian J. Control* 23 (5) (2021) 2185–2200, <http://dx.doi.org/10.1002/asjc.2449>.
- [40] P.T. Boggs, J.W. Tolle, Sequential quadratic programming, *Acta Numer.* 4 (1995) 1–51, <http://dx.doi.org/10.1017/s0962492900002518>.
- [41] J. Mainprice, N. Ratliff, M. Toussaint, S. Schaal, An interior point method solving motion planning problems with narrow passages, in: *2020 29th IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)*, IEEE, 2020, pp. 547–552, <http://dx.doi.org/10.1109/ro-man47096.2020.9223504>.
- [42] J. Kennedy, R. Eberhart, Particle swarm optimization, in: *Proceedings of ICNN'95-International Conference on Neural Networks*, vol. 4, IEEE, 1995, pp. 1942–1948, <http://dx.doi.org/10.1109/ICNN.1995.488968>.
- [43] N. Hansen, The cma evolution strategy: a comparing review, towards a new evolutionary computation: Advances in the estimation of distribution algorithms, 2006, pp. 75–102, [http://dx.doi.org/10.1007/3-540-32494-1\\_4](http://dx.doi.org/10.1007/3-540-32494-1_4).
- [44] S. Das, P.N. Suganthan, Differential evolution: A survey of the state-of-the-art, *IEEE Trans. Evol. Comput.* 15 (1) (2010) 4–31.
- [45] Ö. Ekrem, B. Aksoy, Trajectory planning for a 6-axis robotic arm with particle swarm optimization algorithm, *Eng. Appl. Artif. Intell.* 122 (2023) 106099, <http://dx.doi.org/10.1016/j.engappai.2023.106099>.
- [46] Z. Yang, N. Li, Y. Zhang, J. Li, Mobile robot path planning based on improved particle swarm optimization and improved dynamic window approach, *J. Robot.* 2023 (1) (2023) 6619841, <http://dx.doi.org/10.1155/2023/6619841>.
- [47] X. Yuan, L. Chen, D-pad: Deep-shallow multi-frequency patterns disentangling for time series forecasting, 2024, <http://dx.doi.org/10.48550/arXiv.2403.17814>, arXiv preprint [arXiv:2403.17814](https://arxiv.org/abs/2403.17814).
- [48] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, M. Long, Timesnet: Temporal 2d-variation modeling for general time series analysis, 2022, <http://dx.doi.org/10.48550/arXiv.2210.02186>, CoRR abs/2210.02186.